Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Унарные и бинарные операции над графами»

Выполнили:

студенты группы 25ВВВ3

Шибанов Ф.Д.

Флеонов М.В.

Пенза 2026

Цель работы

Изучить унарные и бинарные операции над графами и реализовать отождествление вершин, стягивание ребра, расщепление вершины, объединение, пересечение, кольцевую сумму и декартово произведение для матриц и списков смежности.

Описание алгоритмов

В работе используются неориентированные помеченные графы с возможными петлями. Матрица смежности симметрична: элемент M[i,j] равен 1 при наличии ребра и 0 при его отсутствии. Единица на главной диагонали обозначает петлю. Кратные ребра отдельно не учитываются.

Список смежности вершины содержит номера всех соседей, соответствующих единицам в строке матрицы. Петля записывается в списке один раз как ссылка на саму вершину. Поэтому число элементов списка при наличии петель не следует отождествлять со степенью вершины, в которой петля учитывается дважды.

При отождествлении две различные вершины заменяются одной, а их соседства объединяются без повторов. Петля у объединенной вершины возникает, если хотя бы у одной исходной вершины была петля либо между выбранными вершинами было ребро. Число вершин уменьшается на единицу.

При стягивании ребра его концы отождествляются, а само стягиваемое ребро удаляется. Петли исходных вершин сохраняются. Операция допустима только для двух различных смежных вершин.

При расщеплении добавляется вершина z, связанная с соседями выбранной вершины v и с самой v. Исходные вершины и ребра сохраняются. Число вершин увеличивается на единицу. Особенности обработки петель зависят от используемого представления и описаны в ходе работы.

Объединение графов содержит вершины и ребра обоих графов; пересечение содержит только общие вершины и общие ребра. Вершины с одинаковыми метками считаются совпадающими. Кольцевая сумма содержит ребра, присутствующие ровно в одном графе; изолированные вершины исключаются.

Вершины декартова произведения G1 × G2 — пары (i,a). Пары (i,a) и (j,b) смежны, если i = j и a смежна с b в G2 либо a = b и i смежна с j в G1. Число вершин равно n1 · n2.

Ход работы

Задания 1 и 1.2*

Задание: сгенерировать две матрицы смежности неориентированных помеченных графов и преобразовать их в списки смежности.

В Program.cs вводится положительное число вершин n. Нечисловой или неположительный ввод запрашивается повторно. Методы GraphMethods.Generate.Matrix и GraphMethods.Generate.GraphList из Library.cs создают два графа одинакового размера и их списки смежности.

Для каждой пары i ≤ j генерируется Random.Shared.Next(2). Значение записывается в M[i,j] и M[j,i], включая диагональ. Для каждой строки создается List<int>, в который добавляются индексы столбцов с единицами. Матрицы и списки выводятся методами GraphMethods.Print. Нумерация вершин начинается с 0.

```
[fedor@olegator lab6_newtry]$ dotnet run
Enter the 1 matrix size (number of vertices): 5

Adjacency matrix of graph G1:
0 0 0 0 0
0 1 1 0 1
0 1 1 0 0
0 0 0 0 1
0 1 0 1 1
Adjacency lists of graph G1:
0:
1: 1, 2, 4
2: 1, 2
3: 4
4: 1, 3, 4

Adjacency matrix of graph G2:
1 1 1 0 0
1 0 1 0 0
1 1 0 1 1
0 0 1 1 1
0 0 1 1 0
Adjacency lists of graph G2:
0: 0, 1, 2
1: 0, 2
2: 0, 1, 3, 4
3: 2, 3, 4
4: 2, 3
```

Рисунок 1 — Матрицы и списки смежности графов G1 и G2

На рисунке 1 графы имеют по пять вершин. В G1 вершина 0 изолирована, список вершины 1 содержит 1, 2 и 4. Запись 1 в собственном списке соответствует петле M[1,1] = 1.

Задание 2.1

Задание: выполнить отождествление вершин, стягивание ребра и расщепление вершины для матричной формы представления графа.

Program.cs последовательно вызывает методы GraphOperations из Library.cs. Все три операции применяются к исходной матрице G1, а не к результату предыдущей операции. Номера вершин вводятся с клавиатуры; допустимый диапазон — от 0 до n − 1. Методы возвращают матрицу результата и признак ошибки err.

Отождествление вершин

1) DefaultCheck проверяет диапазон номеров и различие двух вершин. При ошибке возвращается исходная матрица и устанавливается err = true.

2) Создается матрица размера (n − 1) × (n − 1). Объединенная вершина занимает позицию 0; остальные вершины следуют в исходном порядке без двух выбранных.

3) Связи выбранных вершин переносятся в строку и столбец 0 по правилу логического ИЛИ. Ребра между оставшимися вершинами копируются без изменений. Связи внутри объединяемой пары, включая ее петли, формируют диагональный элемент M′[0,0].

Стягивание ребра

Метод ContractEdge дополнительно проверяет наличие ребра между выбранными вершинами. Затем вызывается IdentifyVertices. Диагональный элемент объединенной вершины устанавливается равным M[u,u] ИЛИ M[v,v]: стягиваемое ребро не образует петлю, но исходные петли сохраняются. При недопустимой операции программа выводит Error.

Расщепление вершины

Метод SplitVertex создает матрицу размера (n + 1) × (n + 1), копирует исходную матрицу, а в последнюю строку и столбец переносит связи вершины v. Между v и новой вершиной с номером n устанавливается ребро. Диагональный элемент новой вершины остается нулевым, даже если у v была петля.

Запуск программы из папки Program: dotnet run. После вывода двух исходных графов пользователь вводит пары вершин для первых двух операций и одну вершину для расщепления.

Результаты матричных операций

```
Task 2.1: operations on G1 (vertex numbers start at 0).

Identify vertices:
First vertex: 1
Second vertex: 4
1 0 1 1
0 0 0 0
1 0 1 0
1 0 0 0

Contract an edge:
First vertex: 1
Second vertex: 4
1 0 1 1
0 0 0 0
1 0 1 0
1 0 0 0

Split a vertex:
Vertex: 1
0 0 0 0 0 0
0 1 1 0 1 1
0 1 1 0 0 1
0 0 0 0 1 0
0 1 0 1 1 1
0 1 1 0 1 0
```

Рисунок 2 — Унарные операции над матрицей смежности G1

В примере с рисунков 1 и 2 отождествляются вершины 1 и 4. Порядок вершин результата: объединенная вершина, затем исходные 0, 2 и 3. Объединенная вершина смежна с 2 и 3 и имеет петлю. Число вершин уменьшается с пяти до четырех.

При стягивании ребра между теми же вершинами получается такая же матрица: у вершин 1 и 4 уже были петли, поэтому петля объединенной вершины сохраняется. При расщеплении вершины 1 добавляется вершина 5, смежная с 1, 2 и 4. Размер матрицы становится 6 × 6; $M'[5,5] = 0$.

Задание 2.2*

Задание: выполнить унарные операции над графом в форме списков смежности. Программа python_part/task2_2.py вводит n от 2 до 10, генерирует матрицу и строит словарь списков с именами v1, …, vn. Перед каждой операцией списки восстанавливаются из исходной матрицы.

Функция identify объединяет соседей u и v без повторов, заменяет ссылки на u и v ссылкой на z и удаляет ключи u и v. Петля z сохраняется при наличии петли у u или v либо ребра между ними. Функция contract сначала удаляет стягиваемое ребро, затем вызывает identify. Функция split копирует соседей в список z и добавляет ребро v–z; петля v при этом дает петлю z.

```
[fedor@olegator python_part]$ python task2_2.py
Enter number of vertices of G1 (2..10): 5

=== Task 2.2* ===
G1 adjacency matrix:
    v1  v2  v3  v4  v5
v1  0   0   0   1   0
v2  0   1   1   1   0
v3  0   1   1   0   0
v4  1   1   0   1   0
v5  0   0   0   0   1
G1 adjacency lists:
v1  : v4 -> NULL
v2  : v2 -> v3 -> v4 -> NULL
v3  : v2 -> v3 -> NULL
v4  : v1 -> v2 -> v4 -> NULL
v5  : v5 -> NULL
a) Identification of vertices. Enter u and v: 1 2
v3  : v3 -> z -> NULL
v4  : v4 -> z -> NULL
v5  : v5 -> NULL
z   : v4 -> v3 -> z -> NULL
b) Contraction of an edge. Enter u and v: 1 2
Vertices are not adjacent, contraction is impossible
c) Splitting of a vertex. Enter v: 1
v1  : v4 -> z -> NULL
v2  : v2 -> v3 -> v4 -> NULL
v3  : v2 -> v3 -> NULL
v4  : v1 -> v2 -> v4 -> z -> NULL
v5  : v5 -> NULL
z   : v4 -> v1 -> NULL
[fedor@olegator python_part]$
```

Рисунок 3 — Унарные операции над списками смежности G1

Отождествление v1 и v2 дает z с соседями v4, v3 и z; петля у z обусловлена петлей v2. Стягивание v1–v2 невозможно, поскольку ребра нет. При расщеплении v1 список z содержит v4 и v1, а ссылки на z появляются у обеих вершин.

Задание 3

Задание: выполнить объединение, пересечение и кольцевую сумму графов. Программа python_part/task3.py вводит размеры от 2 до 10. Для объединения и кольцевой суммы матрицы дополняются нулями до max(n1,n2); затем поэлементно применяются ИЛИ и исключающее ИЛИ. Пересечение вычисляется операцией И на общих вершинах в матрице размера min(n1,n2). Из кольцевой суммы исключаются нулевые строки и соответствующие столбцы; метки сохраняются. Если ребер нет, выводится Empty graph.

```
[fedor@olegator python_part]$ python task3.py
Enter number of vertices of G1 (2..10): 5
Enter number of vertices of G2 (2..10): 6

 Task 3
G1 adjacency matrix:
    v1  v2  v3  v4  v5
v1  1   0   1   0   1
v2  0   0   1   1   1
v3  1   1   0   0   0
v4  0   1   0   0   0
v5  1   1   0   0   1
G2 adjacency matrix:
    v1  v2  v3  v4  v5  v6
v1  0   1   0   1   0   1
v2  1   1   1   0   0   1
v3  0   1   0   1   0   0
v4  1   0   1   0   1   0
v5  0   0   0   1   0   0
v6  1   1   0   0   0   0
a) Union G1 U G2:
    v1  v2  v3  v4  v5  v6
v1  1   1   1   1   1   1
v2  1   1   1   1   1   1
v3  1   1   0   1   0   0
v4  1   1   1   0   1   0
v5  1   1   0   1   1   0
v6  1   1   0   0   0   0
b) Intersection G1 & G2:
    v1  v2  v3  v4  v5
v1  0   0   0   0   0
v2  0   0   1   0   0
v3  0   1   0   0   0
v4  0   0   0   0   0
v5  0   0   0   0   0
c) Ring sum G1 (+) G2:
    v1  v2  v3  v4  v5  v6
v1  1   1   1   1   1   1
v2  1   1   0   1   1   1
v3  1   0   0   1   0   0
v4  1   1   1   0   1   0
v5  1   1   0   1   1   0
v6  1   1   0   0   0   0
[fedor@olegator python_part]$
```

Рисунок 4 — Объединение пересечение и кольцевая сумма графов

В примере n1 = 5, n2 = 6. Объединение имеет шесть вершин. Единственное общее ребро — v2–v3, поэтому оно остается в пересечении и отсутствует в кольцевой сумме. В кольцевой сумме нет изолированных вершин; петли v1, v2, v4 и v5 сохраняются, поскольку каждая встречается ровно в одном исходном графе.

Задание 4*

Задание: построить декартово произведение графов по матрицам смежности. Программа python_part/task4.py вводит n1 и n2 от 2 до 10, генерирует и выводит исходные матрицы.

Вершине (i,a) соответствует индекс i · n2 + a при нумерации индексов с нуля. Четыре вложенных цикла перебирают пары (i,a) и (j,b). Единица записывается, если (i = j и M2[a,b] = 1) либо (a = b и M1[i,j] = 1). Результат выводится с метками пар, компоненты которых начинаются с 1.

```
[fedor@olegator python_part]$ python task4.py
Enter number of vertices of G1 (2..10): 4
Enter number of vertices of G2 (2..10): 3

=== Task 4* ===
G1 adjacency matrix:
    v1  v2  v3  v4
v1  0   1   0   0
v2  1   1   0   1
v3  0   0   0   1
v4  0   1   1   1
G2 adjacency matrix:
    v1  v2  v3
v1  1   1   1
v2  1   0   0
v3  1   0   1
Cartesian product G1 x G2:
        (1,1)   (1,2)   (1,3)   (2,1)   (2,2)   (2,3)   (3,1)   (3,2)   (3,3)   (4,1)   (4,2)   (4,3)
(1,1)   1       1       1       1       0       0       0       0       0       0       0       0
(1,2)   1       0       0       0       1       0       0       0       0       0       0       0
(1,3)   1       0       1       0       0       1       0       0       0       0       0       0
(2,1)   1       0       0       1       1       1       0       0       0       1       0       0
(2,2)   0       1       0       1       1       0       0       0       0       0       1       0
(2,3)   0       0       1       1       0       1       0       0       0       0       0       1
(3,1)   0       0       0       0       0       0       1       1       1       1       0       0
(3,2)   0       0       0       0       0       0       1       0       0       0       1       0
(3,3)   0       0       0       0       0       0       1       0       1       0       0       1
(4,1)   0       0       0       1       0       0       1       0       0       1       1       1
(4,2)   0       0       0       0       1       0       0       1       0       1       1       0
(4,3)   0       0       0       0       0       1       0       0       1       1       0       1
[fedor@olegator python_part]$
```

Рисунок 5 — Декартово произведение графов

На рисунке 5 исходные графы имеют четыре и три вершины. Матрица произведения имеет размер 12 × 12. Например, вершина (1,1) смежна с (1,2), (1,3) по ребрам G2 и с (2,1) по ребру G1. Петля у (1,1) обусловлена петлей первой вершины G2.

Петля у пары (i,a) существует, если петля есть хотя бы у одной из соответствующих вершин исходных графов. Каждый слой с фиксированной первой компонентой воспроизводит связи G2, а связи между слоями определяются ребрами G1.

Результаты работы

Для хранения и обработки графов использованы C# и Python. В C# реализованы генерация матриц и списков смежности, вывод обоих представлений и унарные операции над матрицей. В Python реализованы унарные операции над списками, бинарные операции над матрицами и декартово произведение.

Число вершин после отождествления и стягивания равно n − 1, после расщепления — n + 1. Объединение содержит max(n1,n2) вершин, пересечение — min(n1,n2), кольцевая сумма — только вершины с ненулевыми строками матрицы результата. Декартово произведение содержит n1 · n2 вершин.

Петли учитываются при генерации и выполнении операций. При расщеплении C# оставляет новую вершину без собственной петли, а Python переносит на новую вершину петлю исходной. Эти правила соответствуют реализациям SplitVertex и split.

Для запуска Python-программ из папки Program/python_part используются команды python task2_2.py, python task3.py и python task4.py. Каждая программа самостоятельно генерирует графы и выводит исходные данные перед результатами операций.

Вывод

В ходе лабораторной работы изучены унарные и бинарные операции над неориентированными помеченными графами. Реализованы матричное представление и списки смежности, преобразование матрицы в списки, отождествление вершин, стягивание ребра и расщепление вершины. Объединение, пересечение и кольцевая сумма реализованы с помощью поэлементных логических операций над матрицами смежности. Построено декартово произведение с вершинами в виде упорядоченных пар. Примеры выполнения показывают изменение числа вершин, объединение соседств и обработку петель.